CHAPTER 16 / SUPPLEMENTARY MATERIAL

Cyber–Physical Security and Mission Assurance for Unmanned Aerial Vehicles

Open Challenges and Future Research Directions

Secure AI and Robust Machine Learning for UAV Systems

- **Model integrity:** Verify digital signatures on model updates before deployment and record the model version used for each flight.
- **Robustness testing:** Measure missed detections and false alarms under glare, fog, motion blur, and partial occlusion. Use the results to define where the perception model can operate reliably.
- **Monitoring and fallback:** Monitor prediction confidence and sensor health in flight. When perception degrades, alert the operator and use a tested fallback, such as slowing down or landing at a designated site.

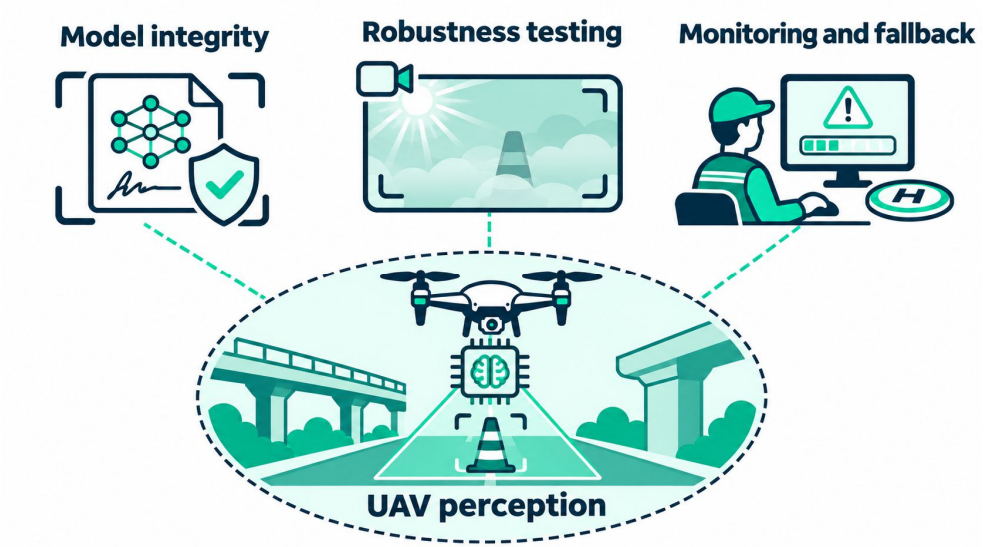


Figure S16.1. An inspection UAV uses onboard machine learning to detect obstacles. Model verification, testing under degraded visibility, and in-flight monitoring address different points at which perception can fail.

Adversarial Robustness in UAV Perception

- **Controlled stress tests:** In simulation, compare the perception model's outputs on unmodified inputs and inputs deliberately altered to mislead it. Keep the scene and ground-truth labels fixed.
- **Measure detection failures:** Record missed obstacles, false detections, and localization errors. Check whether the model remains highly confident when it is wrong.
- **Cross-check and fallback:** Compare camera detections with an independent ranging sensor. Test whether persistent disagreement triggers an operator alert and a predefined fallback.

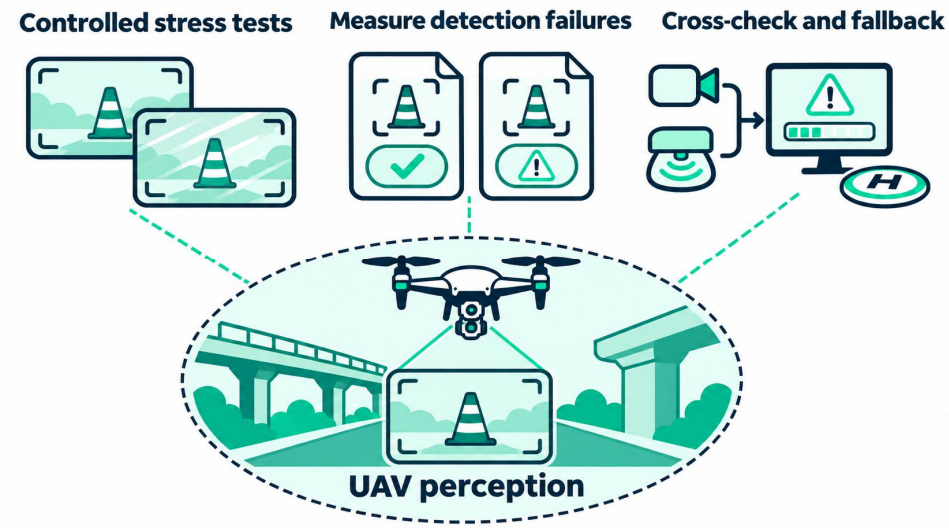


Figure S16.2. Paired simulated scenes show how deliberate input changes can affect UAV obstacle detection. An independent sensor provides a second source of evidence when the camera model's output is unreliable.

Resilient Swarm Intelligence and Distributed Autonomy

- **Local coordination:** Nearby UAVs share position and task status so each aircraft can coordinate its search with its neighbors.
- **Task reassignment:** If a UAV leaves the mission, another can take over its unfinished sector when battery reserve and flight conditions permit.
- **Link-loss fallback:** When coordination messages become stale, the affected UAV follows a predefined contingency, such as returning to a designated recovery point, while the remaining team updates its coverage plan.

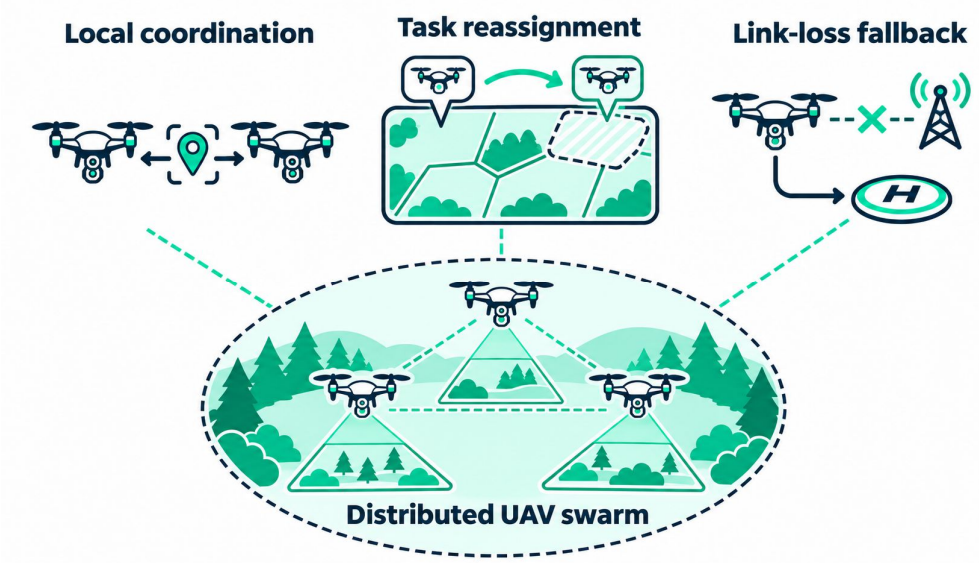


Figure S16.3. Three UAVs divide a forest search area into sectors. If one aircraft loses its communication link, it follows a recovery procedure while the remaining aircraft decide whether they can cover its unfinished sector.

Quantum Threat Model for UAV Missions

- **Data sensitivity lifetime ($T_{sensitivity}$):** Estimate how long mission imagery, telemetry, or flight logs must remain confidential. An adversary could record encrypted traffic today and attempt to decrypt it later.
- **Quantum risk horizon (T_Q):** Use a planning scenario for when a quantum computer might threaten the public-key cryptography protecting mission data. Evaluate several horizons to account for uncertainty.
- **Migration lead time ($T_{migration}$):** Include the time needed to update, test, and deploy cryptographic changes across aircraft, ground stations, and supporting services.

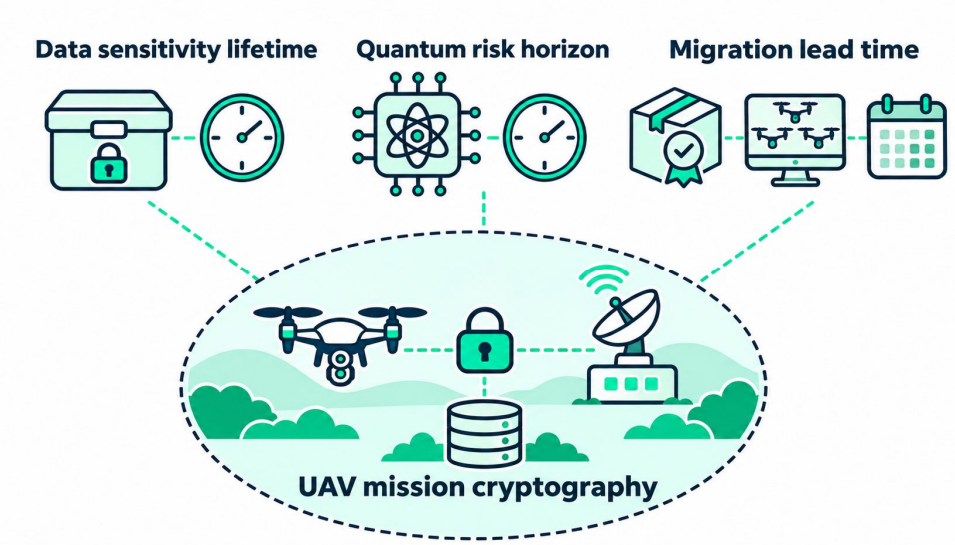


Figure S16.4. Migration becomes urgent when the time needed to update the fleet plus the confidentiality lifetime of newly collected data reaches or exceeds the assumed quantum risk horizon.

Quantum Threat Model for UAV Missions

Exercise — Compare two UAV fleets

Fleet A collects sensitive survey imagery. Fleet B collects routine inspection data and can update its cryptography more easily. All values are hypothetical. Use T_Q=8 years for both fleets.

Input	Fleet A	Fleet B
Data sensitivity lifetime ($T_{sensitivity}$) (years)	6	3
Migration lead time \($T_{migration}$) (years)	3	2
Mission criticality ($C_{mission}$)	4	3
Exposure ($E_{exposure}$)	4	2
Deployment difficulty ($D_{deployment}$)	3	2
Cryptographic agility ($A_{agility}$)	2	4

The last four inputs use a 1–5 scale, with 5 indicating the highest level. For the exercise, set (α=1) point per year and (ϐ=ϒ=δ=η=1\) point per rating unit.

1. Apply Eq. (16.53), ($T_{sensitivity}+T_{migration}>T_Q$, to identify which fleet meets the migration urgency criterion.

2. Compute the relative priority score using Eq. (16.54).

3. Recheck the timing criterion if Fleet A shortens migration from three years to two, with all other inputs fixed.

Worked answer.
Fleet A meets the criterion because (6+3=9>8). Fleet B falls below it because (3+2=5<8). Fleet A also ranks higher on the relative priority score, with (Q_r=15) versus (Q_r=6). Shortening Fleet A's migration to two years brings it exactly to the urgency threshold ((6+2=8)). Its priority score remains 15 because migration lead time does not enter Eq. (16.54) directly.

Open Challenge 1: Resource Constraints and Real-Time Performance

- **Onboard resources:** Benchmark PQC key encapsulation, signing, and verification on the target hardware. Record execution time, peak RAM, and energy per handshake while flight software is running.
- **Link overhead:** Count the bytes exchanged for public keys, ciphertexts, signatures, certificates, and metadata. Estimate transmission time using the effective data rate available to the handshake.
- **Latency Budget:** Compare total handshake time with the mission deadline. Test under representative processor load and link conditions, including packet loss and retransmissions.

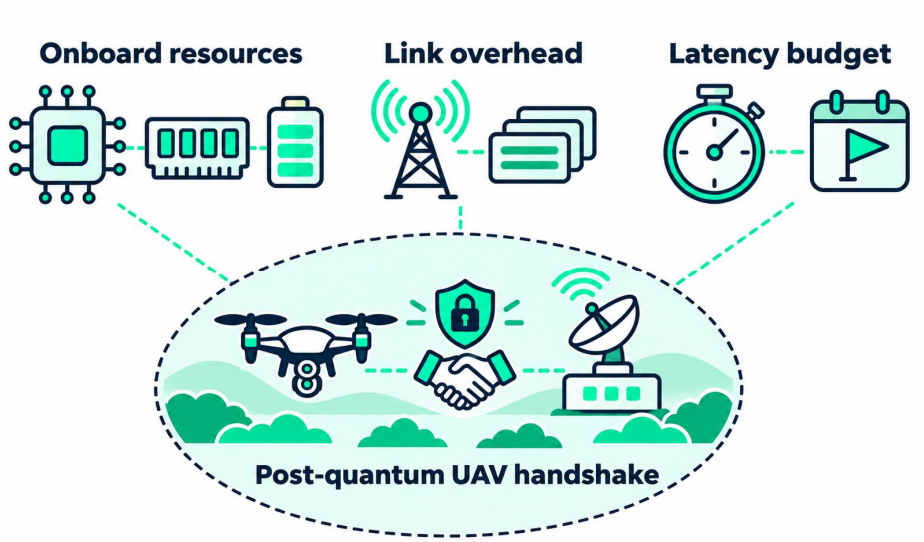


Figure S16.5. A post-quantum UAV handshake draws on both onboard computing resources and radio capacity. Either can determine whether the handshake meets the mission deadline.

Open Challenge 1: Resource Constraints and Real-Time Performance

Worked exercise — Can the handshake meet a 250 ms deadline?

A UAV and its ground station establish a secure session. Use these **hypothetical** inputs and assume processing and transmission occur sequentially.

Input	Value
KEM processing, (T_{KEM})	18 ms
Signing, (T_{SIG})	24 ms
Signature verification, (T_{VER})	12 ms
Key derivation, (T_{KDF})	2 ms
Public key, ciphertext, signature, certificate, metadata	1,200 + 1,100 + 2,400 + 1,500 + 200 bytes
Effective link rate, (R_{link})	500 kb/s nominal, 250 kb/s degraded
Maximum handshake latency, (T_{max})	250 ms

Open Challenge 1: Resource Constraints and Real-Time Performance

1. Use Eq. (16.57) to calculate transmission time at each link rate. Convert bytes to bits before dividing by the rate.
2. Use Eq. (16.55) to calculate total handshake latency, then apply Eq. (16.56) to check the deadline.
3. Find the minimum effective link rate needed to meet the deadline under this model.

Worked answer. The exchange totals (6400) bytes, or (51200) bits. Processing takes (18+24+12+2=56) ms.

Effective link rate	Transmission time	Total handshake time	Deadline margin
500 kb/s	102.4 ms	158.4 ms	91.6 ms to spare
250 kb/s	204.8 ms	260.8 ms	10.8 ms over

The 250 ms deadline leaves (250-56=194) ms for transmission. The minimum modeled rate is therefore (51200/0.194≈264) kb/s. At the degraded rate, transmission alone takes 204.8 ms, leaving just 45.2 ms for processing against the 56 ms required.

Open Challenge 2: Energy Consumption and Flight Endurance

- **Processor and memory:** Measure the extra energy consumed by the UAV's processor and memory during post-quantum key establishment and authentication.
- **Radio transmisión:** Account for the time the onboard radio spends transmitting handshake messages. A lower link rate or retransmissions can increase that time.
- **Battery reserve:** Add the energy of all expected handshakes and rekeying events to the mission budget. Check whether the planned flight still preserves the required reserve.

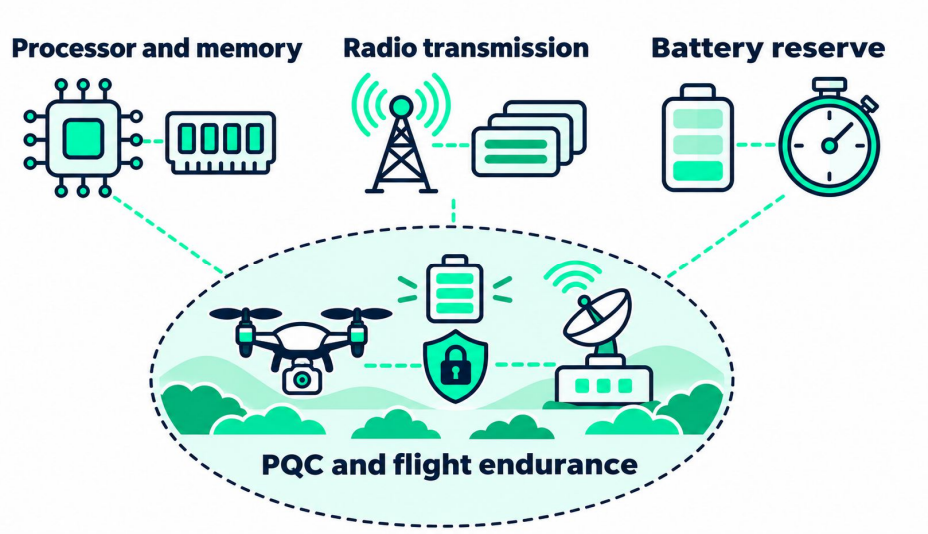


Figure S16.6. The energy cost of a post-quantum handshake depends on onboard processing, memory activity, and radio transmission. Repeated handshakes increase the total energy drawn from the mission budget.

Open Challenge 2: Energy Consumption and Flight Endurance

Worked exercise — How much flight time does rekeying cost?

A survey UAV completes one initial post-quantum handshake and then rekeys every five seconds. Assume each rekey has the same energy cost as the initial handshake. The values are hypothetical, and the power figures represent **additional onboard draw** above the baseline mission load.

Input	Value
CPU power and active time	(3.0 W), (0.060 s)
Radio power and transmission time	(2.5 W), (0.200 s)
Memory power and active time	(0.5 W), (0.100 s)
Baseline flight, sensing, and compute power	(120 W)
Battery energy and required reserve	(60 Wh), (12 Wh)

Open Challenge 2: Energy Consumption and Flight Endurance

1. Use Eq. (16.58) to calculate the energy per handshake in joules.
2. For a 24-minute flight, calculate the total handshake energy and use Eq. (16.59) to check whether the battery reserve is preserved. Count the initial handshake and rekeys at (5,10,….,1435) seconds.
3. Estimate the maximum flight duration using the average rekeying power.

Worked answer

1. **Energy per handshake.** (3.0(0.060)+2.5(0.200)+0.5(0.100)=0.73 J).
2. **Reserve check.** The 288 exchanges consume (288(0.73)=210.24 J≈0.0584 Wh). The baseline flight consumes (120(24/60)=48 Wh), which uses the full (60-12=48 Wh) available for the mission. Including PQC raises the total to (48.0584 Wh), exceeding the budget.
3. **Flight endurance.** At one exchange every five seconds, the average extra power is (0.73/5=0.146 W). Estimated endurance is (48/(120+0.146)≈23.97) minutes, about **1.75 seconds shorter** than the 24-minute baseline.

Open Challenge 3: Crypto-Agility and Long-Lived UAV Fleets

- **Algorithms and protocols:** Design aircraft and ground stations to adopt post-quantum key establishment and signatures. Test interoperability between hardware generations before changing the fleet's cryptographic policy.
- **Keys and certificates:** Plan how to provision, rotate, and revoke credentials across the fleet, including aircraft that connect only intermittently.
- **Firmware updates:** Stage signed updates by aircraft model. Verify that each model can authenticate the update and recover securely if installation is interrupted.

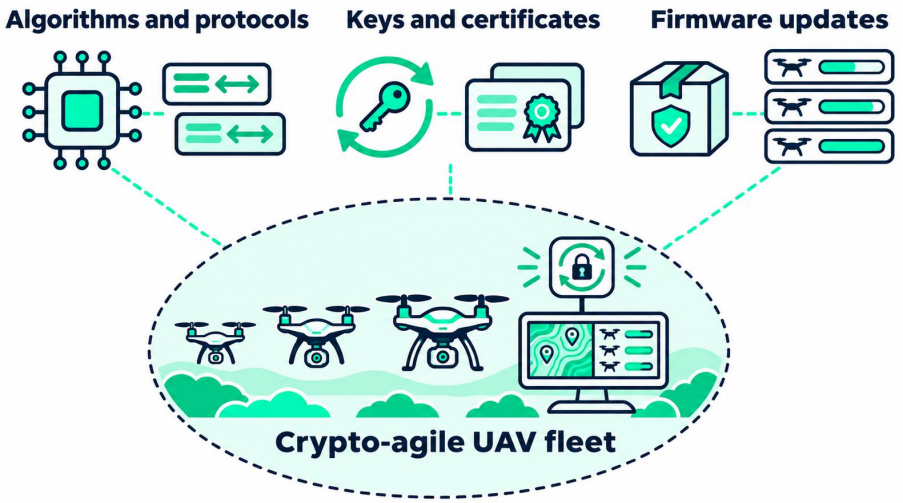


Figure S16.7. A mixed-generation UAV fleet migrates to post-quantum cryptography through coordinated changes to algorithms, protocols, credentials, and firmware.

Open Challenge 3: Crypto-Agility and Long-Lived UAV Fleets

Worked exercise — Which upgrade improves crypto-agility most?

Use Eq. (16.60) to compare a legacy fleet with a newer fleet. The scores and weights are hypothetical. Rate each capability from **0** (unsupported) to **1** (fully supported and tested across the fleet). The weights sum to 1.

Capability	Weight	Legacy fleet	Newer fleet
Algorithm replacement (A_{alg})	0.25	0.30	0.80
Key management (A_{key})	0.20	0.60	0.80
Protocol Migration (A_{proto})	0.20	0.40	0.70
Secure firmware updates (A_{fw})	0.25	0.20	0.90
Certificate management (A_{cert})	0.10	0.50	0.70

Open Challenge 3: Crypto-Agility and Long-Lived UAV Fleets

1. Calculate the crypto-agility index for each fleet.
2. For the legacy fleet, compare two equal-cost upgrades. One raises firmware agility from 0.20 to 0.70. The other raises certificate agility from 0.50 to 1.00. Hold all other ratings fixed.
3. Identify one interoperability test the legacy fleet should pass before deploying PQC.

Worked answer. The legacy fleet scores **0.375** and the newer fleet scores **0.795**. The firmware upgrade adds $(0.25(0.50)=0.125)$, raising the legacy score to **0.500**. The certificate upgrade adds $(0.10(0.50)=0.050)$, raising it to **0.425**. Firmware therefore gives the larger gain under these weights. Before rollout, test whether the oldest supported aircraft can establish an authenticated session with the updated ground station using the intended post-quantum configuration.

Open Challenge 4: Secure Boot, Firmware Signing, and Supply-Chain Trust

- **Firmware signing:** Sign each approved UAV firmware release with a post-quantum signature. Protect the signing key and record which build and components were approved for release.
- **Secure boot verification:** Before executing firmware, verify its signature using a public key anchored in the device's root of trust. Measure verification time, peak RAM, and flash usage on the target controller.
- **Supply-chain trust:** Track firmware provenance from supplier components to the signed release. A valid signature authenticates the image under the trusted key. Component review and release approval address whether the image should have been signed.

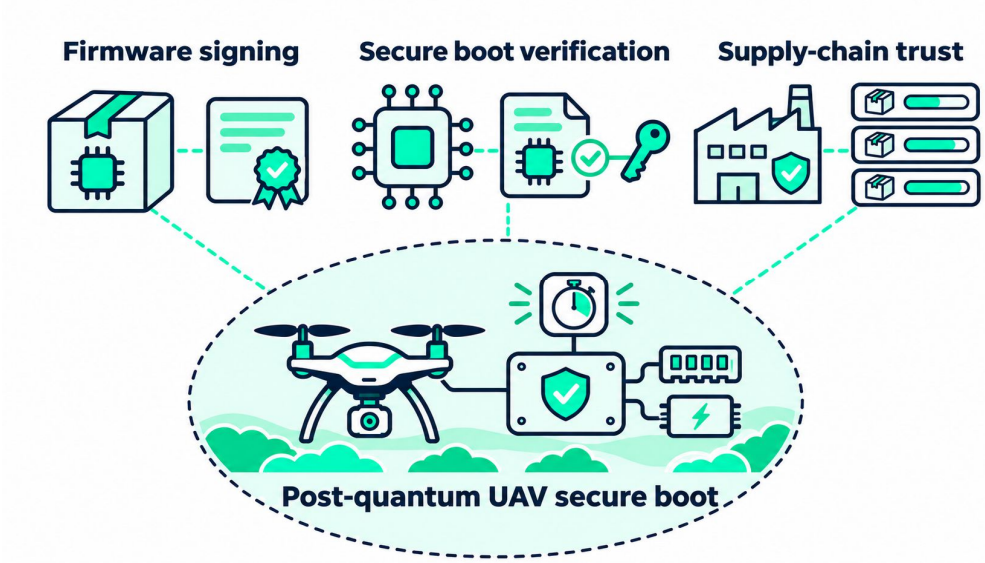


Figure S16.8. A UAV bootloader checks the firmware's post-quantum signature before execution. The verification must fit within the controller's time, RAM, and flash budgets.

Open Challenge 4: Secure Boot, Firmware Signing, and Supply-Chain Trust

Worked exercise — Can a legacy UAV controller support PQC secure boot?

A flight controller has two candidate post-quantum verifiers. Use the hypothetical measurements below. Assume the flash reserve is dedicated to the signature and public key.

Quantity	Controller limit	Candidate A	Candidate B
Verification time (T_{verify})	100 ms	78 ms	94 ms
Peak verification RAM (M_{verify})	48 KiB	52 KiB	42 KiB
Signature size (S_{sig})	—	3.3 KiB	2.4 KiB
Public-key size (S_{pk})	—	1.9 KiB	1.3 KiB
Combined signature and key storage	5.0 KiB	Calculate	Calculate

Open Challenge 4: Secure Boot, Firmware Signing, and Supply-Chain Trust

1. Use Eq. (16.62) to determine which candidate satisfies all three resource constraints.
2. Apply Eq. (16.61) to the candidate that fits. What is Boot for an approved image with a valid signature? What is Boot if the image is modified after signing and verification fails?
3. How much verification time, RAM, and flash headroom remains for the candidate that fits?

Worked answer. Candidate A meets the time limit but exceeds the RAM budget by **4 KiB** and the flash reserve by **0.2 KiB** because its signature and key total (3.3+1.9=5.2) KiB. Candidate B fits all three budgets, with **6 ms**, **6 KiB of RAM**, and **1.3 KiB of flash** to spare. For Candidate B, Eq. (16.61) gives (Boot=1) when the signature verifies under the trusted key and (Boot=0) when verification fails.

Open Challenge 5: Swarm Authentication and Group Key Management

- **Member authentication.** Verify each UAV's identity and mission membership before admitting it. Use post-quantum authentication and key establishment to protect the exchange that gives authorized members access to the group key.
- **Group key establishment.** A full mesh requires \(N(N-1)/2\) pairwise secure channels. A shared symmetric group key can protect group traffic, while individual credentials support sender attribution.
- **Rekey on membership change.** When a UAV joins or leaves, establish a fresh key for the updated membership. Keep earlier keys from new members and future keys from departing members.

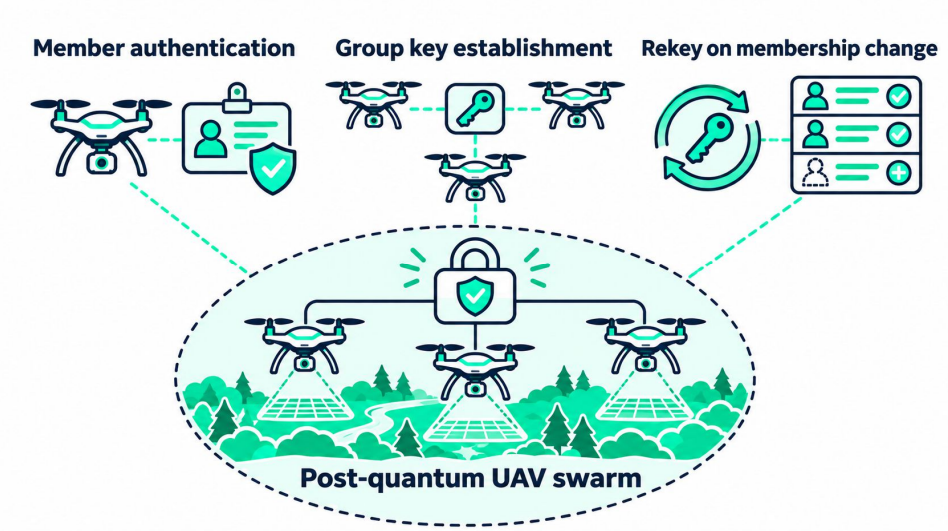


Figure S16.9. A civilian mapping swarm authenticates each UAV before granting access to the group key. Membership changes trigger rekeying so that only current members can access future group traffic.

Open Challenge 5: Swarm Authentication and Group Key Management

Worked exercise — How does setup cost scale?

A civilian mapping swarm grows from **6 to 12 UAVs**. Each pairwise post-quantum handshake costs a hypothetical (CKEM=20) mJ, measured across both participating endpoints.

1. Use Eqs. (16.63) and (16.64) to calculate the number of pairwise channels and the total initialization energy for each swarm size.
2. For the 12-UAV swarm, compare the full mesh with a design in which one UAV acts as coordinator and establishes one secure channel with each of the other 11 UAVs. Calculate the coordinator-based **channel setup energy**, then identify an additional cost to measure.
3. In Eq. (16.65), explain the roles of the secret inputs (K_i), the group identifier ID_G), and the epoch (t). What must change if a member leaves?

Open Challenge 5: Swarm Authentication and Group Key Management

Swarm configuration	Secure channels	Channel setup energy
6 UAVs, full mesh	(6(5)/2=15)	300 mJ
12 UAVs, full mesh	(12(11)/2=66)	1,320 mJ
12 UAVs, coordinator-based	(12-1=11)	220 mJ

Doubling the swarm size increases full-mesh setup energy by **4.4×**. The coordinator-based figure covers pairwise channel setup. Measure group-key distribution, acknowledgment, and retransmission energy separately to estimate total initialization cost.

In Eq. (16.65), the (K_i) provide secret keying material, (ID_G) binds the derivation to the group, and (t) identifies the key epoch. After a member leaves, advance the epoch and establish the next key using fresh secret material unavailable to that UAV. Changing the public epoch alone leaves the new key computable by anyone who still knows all the secret inputs.

Open Challenge 6: Interoperability with 5G, 6G, NTN, and Satellite Links

- **Terrestrial 5G and future 6G:** Test whether the UAV and ground station can preserve or re-establish an authenticated session when the aircraft changes networks.
- **Non-terrestrial networks (NTN) and satellite links:** Measure how available bandwidth, round-trip delay, and intermittent coverage affect post-quantum handshakes and rekeying.
- **Overhead and packet loss:** Estimate the useful payload rate after accounting for security overhead and packet loss. Evaluate handover interruption separately against the mission's timing requirements.

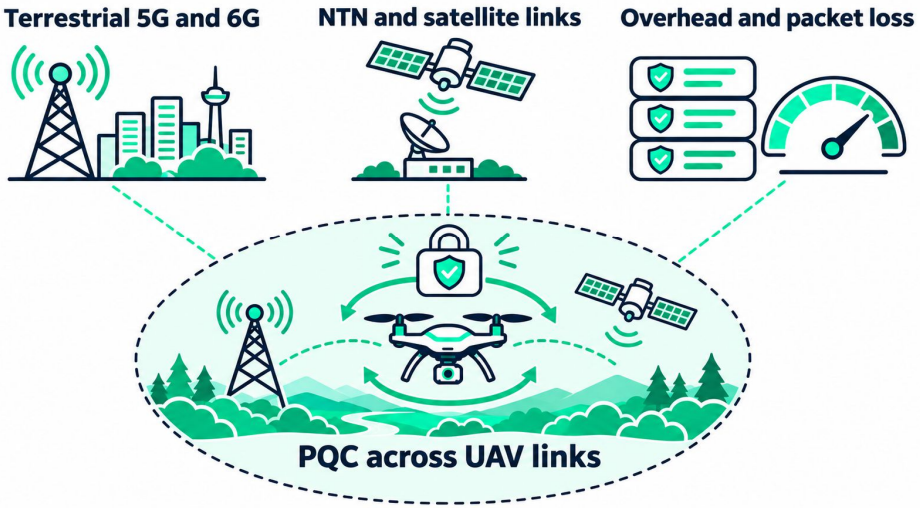


Figure S16.10. A civilian mapping swarm authenticates each UAV before granting access to the group key. Membership changes trigger rekeying so that only current members can access future group traffic.

Open Challenge 6: Interoperability with 5G, 6G, NTN, and Satellite Links

Worked exercise — Can telemetry continue after a satellite handover?

A UAV switches from terrestrial 5G to a satellite NTN link. Its telemetry stream needs **at least 300 kb/s** of useful payload throughput. Use the hypothetical inputs below. Treat ($S_{PQC,ovh}$) as the average security overhead per payload unit, including handshake and rekeying bytes spread across the payload units they protect.

Input	Terrestrial 5G	Satellite NTN
Link rate (R_{link})	1,000 kb/s	400 kb/s
Payload size ($S_{payload}$)	1,000 bytes	1,000 bytes
Average PQC overhead ($S_{PQC,ovh}$)	200 bytes	200 bytes
Packet loss (P_{loss})	2%	12%

Open Challenge 6: Interoperability with 5G, 6G, NTN, and Satellite Links

1. Use Eq. (16.66) to estimate the useful payload rate on each link.
2. Determine whether the telemetry stream meets its throughput requirement after handover.
3. At the satellite link rate and loss rate shown, find the maximum average security overhead that still meets the requirement. Name one handover metric this calculation leaves out.

Worked answer. The payload fraction is (1000/(1000+200)=5/6). The useful payload rates are

$$R_{5G} = 1{,}000(5/6)(0.98) = 816.7\ \text{kb/s}, \qquad R_{NTN} = 400(5/6)(0.88) = 293.3\ \text{kb/s}.$$

The satellite link falls **6.7 kb/s short**. Solving (400(0.88)\,[1000/(1000+$S_{PQC,ovh}$)]>300) gives a maximum average overhead of **173.3 bytes per payload unit**. Eq. (16.66) estimates steady-state throughput. Measure the **handover interruption time** separately, including any delay from re-establishing the secure session.

Open Challenge 7: Side-Channel and Fault-Injection Resistance

- **Leakage assessment:** Evaluate whether observable timing, power consumption, or electromagnetic emissions depend on secret key material. In Eq. (16.67), the observation (L(t)) depends on the secret key Ksec), internal computation state (X(t)), and measurement noise (N(t)).
- **Hardened implementation:** Use constant-time routines to reduce timing leakage and evaluate physical leakage separately on the target hardware. Equation (16.68) expresses the goal that observations reveal negligible information about the secret key.
- **Fault detection and recovery:** Check critical cryptographic results before accepting them. If a check fails, reject the result and follow the defined recovery procedure.

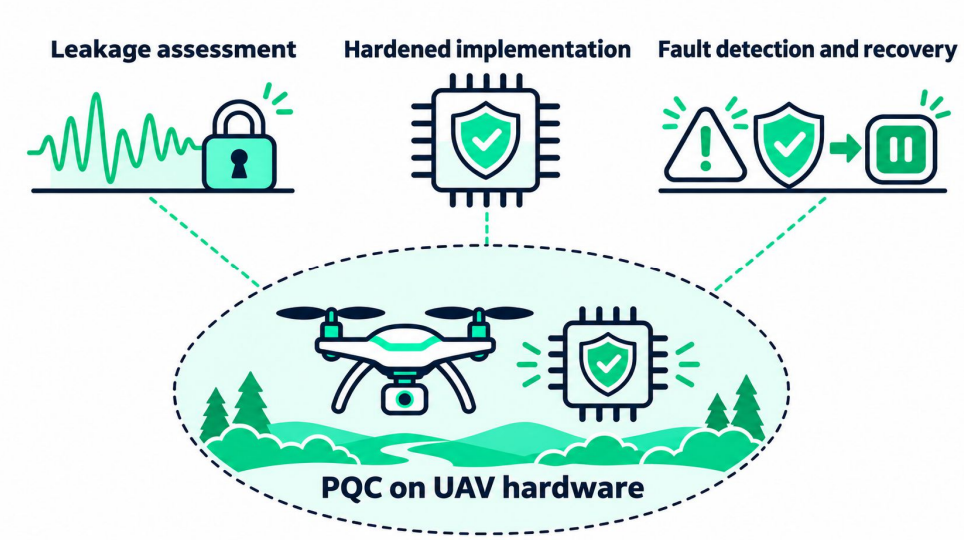


Figure S16.11. Post-quantum cryptography on UAV hardware must protect secrets during normal execution and reject invalid results when a computation is disturbed.

Open Challenge 7: Side-Channel and Fault-Injection Resistance

Worked exercise — Does the observed signal reveal information?

Use a **synthetic** version of Eq. (16.67). Let (K) be a uniformly random dummy secret bit and (L) a binary observation derived from an abstract leakage signal. The table gives the probability that (L=1) for each value of (K).

Dummy secret	Baseline implementation	Hardened implementation
(K=0)	0.20	0.50
(K=1)	0.80	0.50

1. Find the marginal probability (P(L=1)) for each implementation.
2. Calculate I(K;L)=H(L)-H(L|K) in bits. Use the binary entropy h2(p)=-p log_2p-(1-p)\log_2(1-p).
3. Compare both results with Eq. (16.68). In a separate simulated fault test, the verifier reports an internal inconsistency. What should happen to its output?

Worked answer: Both implementations give P(L=1)=0.50, so H(L)=1 bit. For the baseline, H(L|K)=[h_2(0.20)+h_2(0.80)]/2 ≈0.722 bits, giving **I(K;L)≈0.278 bits**. For the hardened case, H(L|K)=1 bit, giving **I(K;L)=0** for this synthetic observation. The matching marginal probabilities hide the baseline's dependence on the secret. In the fault test, discard the verifier's output and follow the defined recovery procedure before accepting any new result.

Open Challenge 8: Certification, Safety Assurance, and Operational Integration

- **Certification evidence:** Link each safety requirement affected by the post-quantum cryptography (PQC) migration to a test result for the specific hardware, firmware, and protocol configuration.
- **Mission safety limits:** Verify that cryptographic operations meet their deadlines, fit in available memory, stay within the energy budget, and maintain the required link reliability under representative conditions.
- **Operational integration:** Test aircraft and ground stations together before rollout. Exercise rekeying under degraded connectivity and verify the recovery procedure if the secure session cannot be restored within the mission deadline.

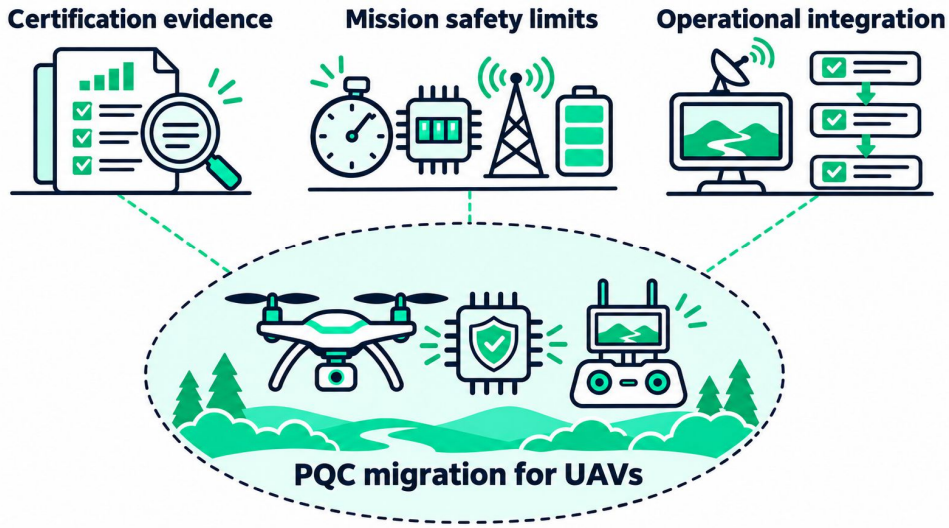


Figure S16.12. A UAV fleet's PQC migration combines traceable test evidence, mission-level performance checks, and a staged rollout across aircraft and ground stations.

Open Challenge 8: Certification, Safety Assurance, and Operational Integration

Worked exercise — Which PQC configuration meets the mission limits?

A survey UAV undergoes a simulated rekeying test with a degraded link and concurrent flight software load. Use the hypothetical results below. For this exercise, (P_{link}) is the fraction of required messages delivered before their individual deadlines during the test window.

Metric	Mission limit	Configuration A	Configuration B
Rekey latency (T_{crypto})	≤ 200 ms	165 ms	185 ms
Peak crypto RAM (M_{crypto})	≤ 64 KiB	58 KiB	60 KiB
On-time link delivery (P_{link})	≥ 99.0%	98.7%	99.4%
Onboard energy per rekey (E_{crypto})	≤ 1.20 J	0.95 J	1.10 J

Open Challenge 8: Certification, Safety Assurance, and Operational Integration

1. Apply Eq. (16.69). Which configuration meets all four constraints?
2. Calculate the passing configuration's margin against each limit.
3. Repeat the decision if the mission raises its minimum on-time delivery requirement to **99.5%**. Identify one item of evidence to collect before rollout.

Worked answer: Configuration A misses the delivery requirement by **0.3 percentage points**. Configuration B meets all four limits, with **15 ms**, **4 KiB**, **0.4 percentage points**, and **0.10 J** of margin, respectively. At a 99.5% delivery requirement, neither configuration passes. For the assurance review, retain a timestamped rekeying test report that records the exact hardware and firmware versions, the degraded-link conditions, and the number of messages used to calculate the delivery rate.

Digital Twins, Simulation, and Benchmarking Frameworks

- **Calibrate the twin:** Use normal flight logs to tune the UAV, sensor, and communication models. Check that the twin reproduces baseline behavior on a separate set of flights.
- **Simulate attack effects:** In an isolated simulator, represent observable effects such as missing telemetry, inconsistent position reports, or interrupted command delivery. Include ordinary link and sensor faults to see whether the detector raises false alarms under similar conditions.
- **Benchmark resilience:** Run each defense against the same mission scenarios and disturbance profiles. Report detection latency, false alarms, trajectory error, recovery time, and whether the UAV completed the mission or entered its planned fallback.

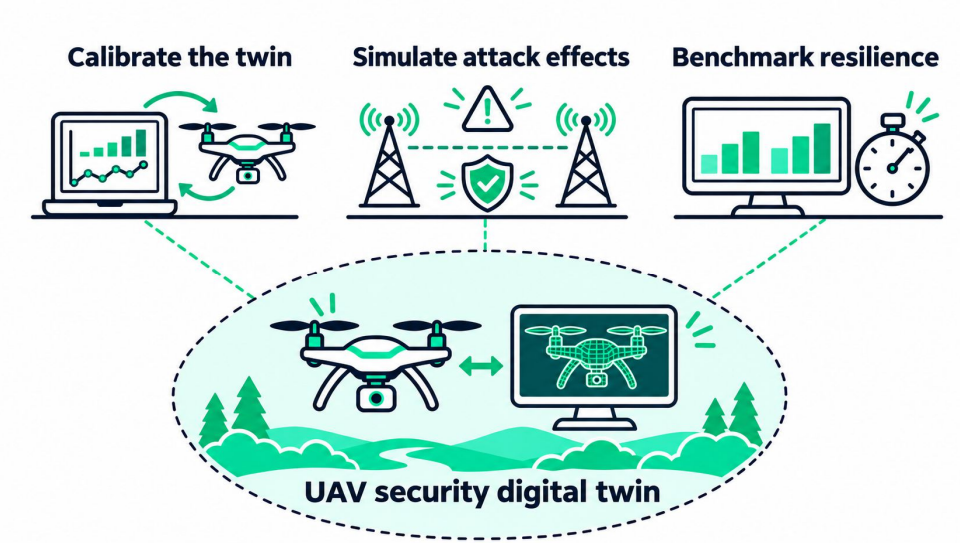


Figure S16.13. A telemetry-calibrated UAV digital twin lets researchers compare how defenses detect simulated disruptions and how the aircraft recovers under repeatable test conditions.

Open Challenge 1: Bridging Simulation and Operational Reality

- **Simulation baseline:** Record how the simulation-trained UAV policy performs on a defined route. Keep the policy fixed when comparing simulation and field results.
- **Operational validation:** Repeat the route in field trials and log wind, sensor quality, and communication delays. Use the same reference trajectory and performance metric in both environments.
- **Measure the discrepancy:** Compute the field loss minus the simulated loss for each matched trial. Report the average gap alongside the individual results so that a large discrepancy in one condition remains visible.

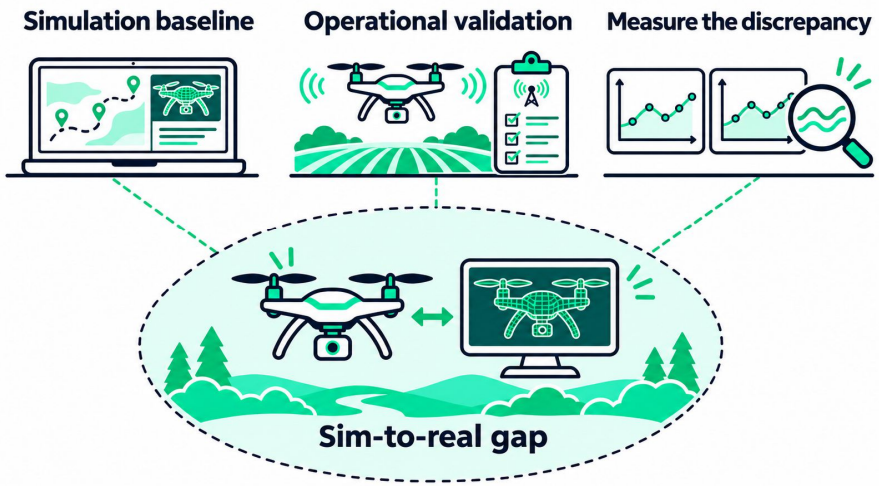


Figure S16.14. The same UAV policy flies a survey route in a digital twin and in the field. Comparing trajectory errors across matched trials shows where the simulation underestimates operational error.

Open Challenge 2: Cyber–Physical Attack Modeling in Digital Twins

- **Model the disruption:** Represent observable attack effects in the digital twin, such as delayed telemetry or inconsistent position reports. Include ordinary link and sensor faults that produce similar symptoms.
- **Track physical effects:** Compare each disrupted run with a matched baseline. Measure how the disruption affects the UAV's trajectory, minimum obstacle clearance, and ability to complete the mission.
- **Evaluate recovery:** Compare defenses under the same simulated conditions. Record detection latency, false alarms during ordinary faults, and whether the UAV resumes its mission or completes the planned fallback.

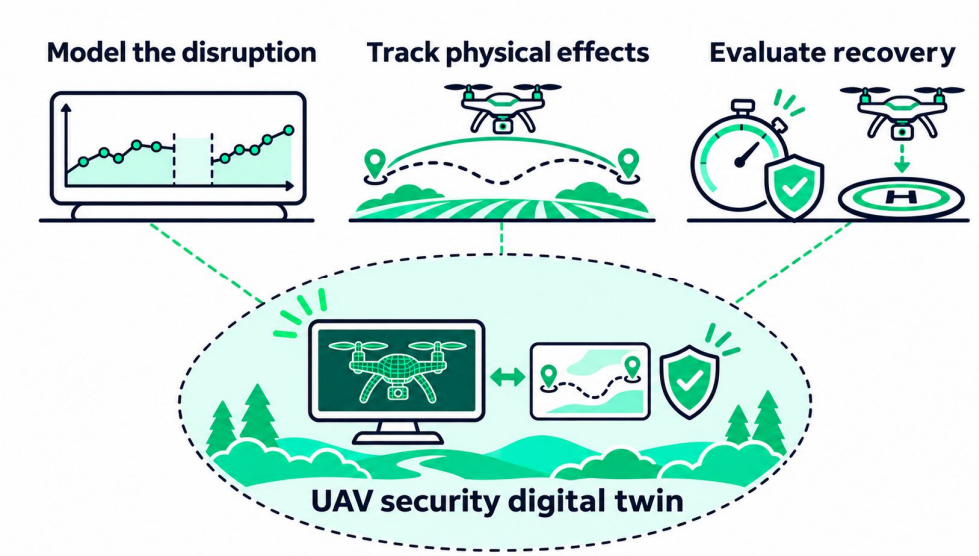


Figure S16.15. A UAV digital twin connects a simulated telemetry disruption to changes in flight behavior, allowing researchers to benchmark detection and recovery under repeatable conditions.

Open Challenge 3: Standardized Benchmark Scenarios

- **Shared scenarios:** Define the UAV mission, route, environmental conditions, and expected fallback behavior. Include nominal flights, ordinary sensor and link faults, and simulated security disruptions.
- **Repeatable test conditions:** Use the same initial state, disturbance schedule, and random seeds when comparing defenses. Record simulator, aircraft model, and software versions so others can reproduce the runs.
- **Common metrics:** Use consistent definitions for detection latency, false alarms, trajectory error, and recovery time. Report mission completion and successful fallback separately so a safe abort is distinguishable from a completed mission.

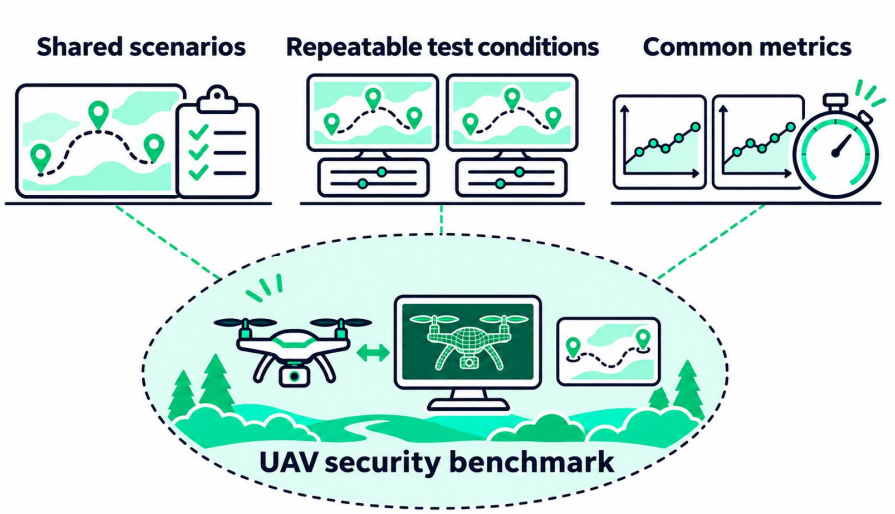


Figure S16.16. A shared UAV digital-twin benchmark gives different defenses the same mission, test conditions, and scoring rules, so differences in detection and recovery can be compared directly.

Open Challenge 4: Unified Metrics for Security, Safety, and Mission Performance

- **Security metrics:** Measure detection rate, false alarms per flight hour, and time from the start of a simulated disruption to the first alert.
- **Safety metrics:** Track minimum obstacle clearance, violations of flight limits, and whether the UAV completes its planned fallback.
- **Mission performance:** Report mission completion rate, survey coverage, and energy consumed. Record how much of the task was completed before any abort.

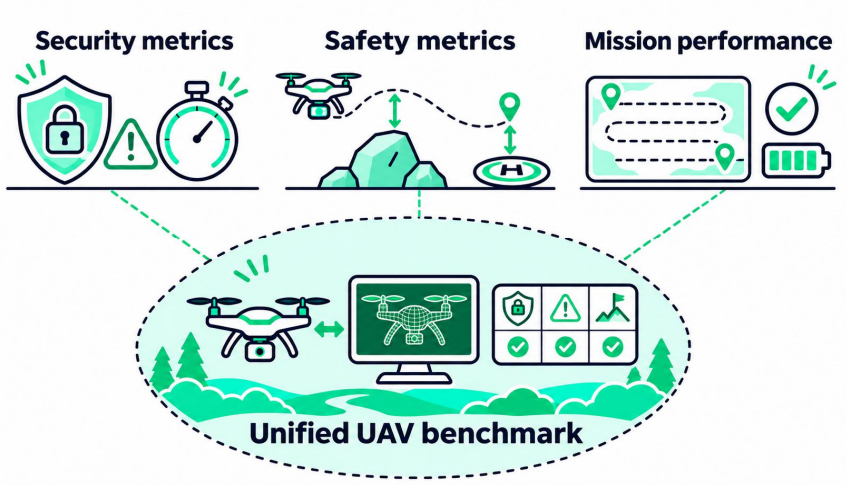


Figure S16.17. A unified UAV benchmark shows whether a defense detects a simulated disruption in time, keeps the aircraft within safety limits, and preserves enough capability to complete the mission.

Open Challenge 5: Reproducibility and Dataset Availability

- **Accessible datasets:** Share time-aligned UAV telemetry, ground-truth trajectories, and event labels. Document units, sampling rates, and access terms. Clearly label synthetic data and explain which field conditions it represents.
- **Versioned test setup:** Release scenario files, simulator and firmware versions, random seeds, and scoring scripts so another team can recreate the test conditions.
- **Independent reruns:** Ask a second team to rerun the benchmark and compare detection latency, trajectory error, and recovery outcomes against the published results using predefined tolerances.

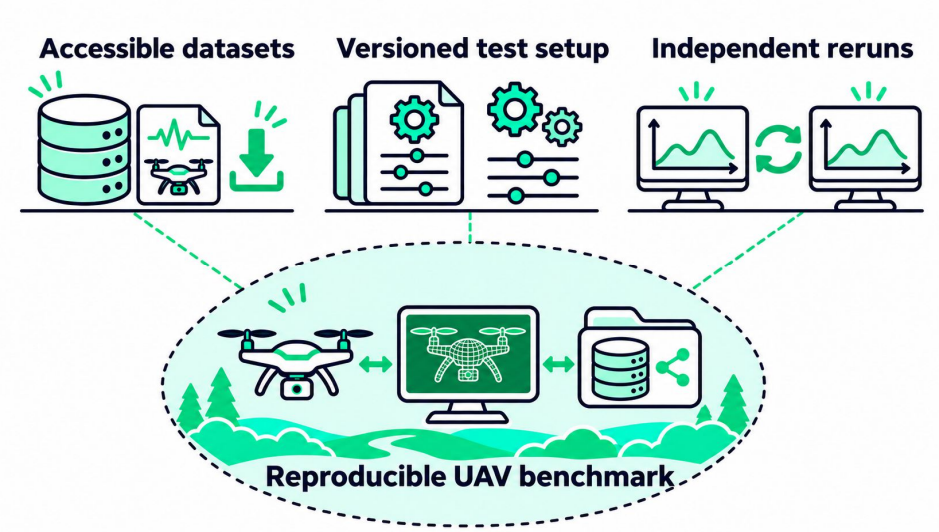


Figure S16.18. A reproducible UAV benchmark pairs an accessible dataset with a versioned digital-twin setup so independent teams can rerun the same scenarios and verify the reported results.

Open Challenge 6: Digital Twins for Swarms and Multi-Domain Robotics

- **Model each platform:** In a flood-response scenario, UAVs map inundated areas, a ground rover checks access routes, and a surface robot surveys waterways. Represent each platform's motion, sensors, and communication limits in the digital twin.
- **Synchronize time and state:** Put each robot's position, sensor observations, and task status on a shared timeline and map. Track the age of each update so delayed information does not appear current.
- **Benchmark team resilience:** Replay the same mission under normal conditions, intermittent links, and the simulated loss of one platform. Compare coverage, time to reassign tasks, and whether each robot stays within its safety limits.

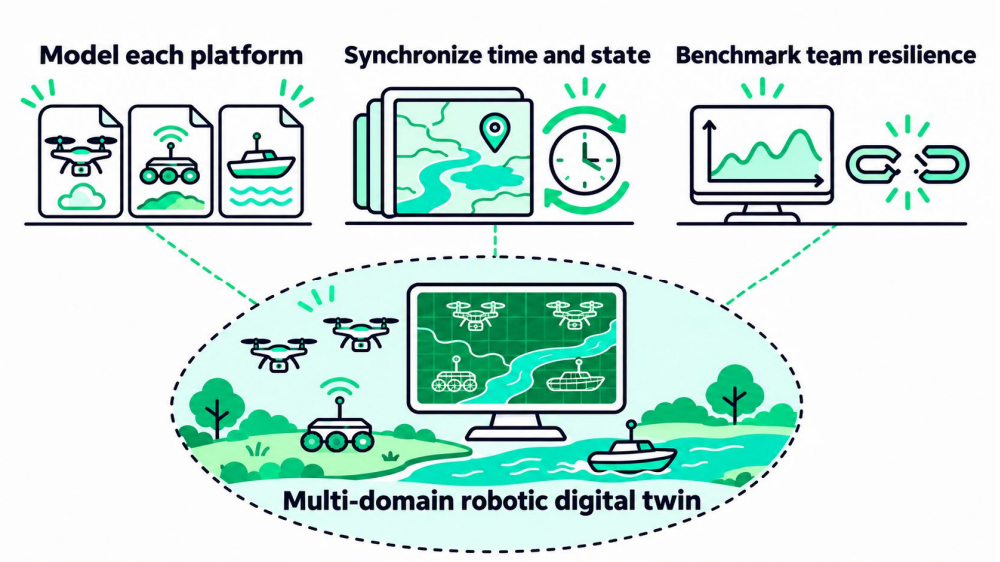


Figure S16.19. A shared digital twin models aerial, ground, and surface robots during flood response. Repeatable scenarios show how communication delays or the loss of one robot affect team coordination and mission coverage.

Open Challenge 7: Benchmarking AI-Enabled UAV Autonomy

- **Test perception:** Compare obstacle detections against simulator ground truth under clear conditions, glare, and partial occlusion. Report missed detections, false alarms, and detection latency.
- **Test decision-making:** Close a survey sector during a simulated mission. Check whether the UAV replans within the allowed time, stays inside the permitted flight area, and initiates its planned fallback if no valid route is available.
- **Measure mission outcomes:** Compare autonomy policies on the same routes, initial conditions, and random seeds. Report mission completion, minimum obstacle clearance, energy use, and safe aborts. Check that the digital twin reproduces relevant flight behavior using independent flight logs.

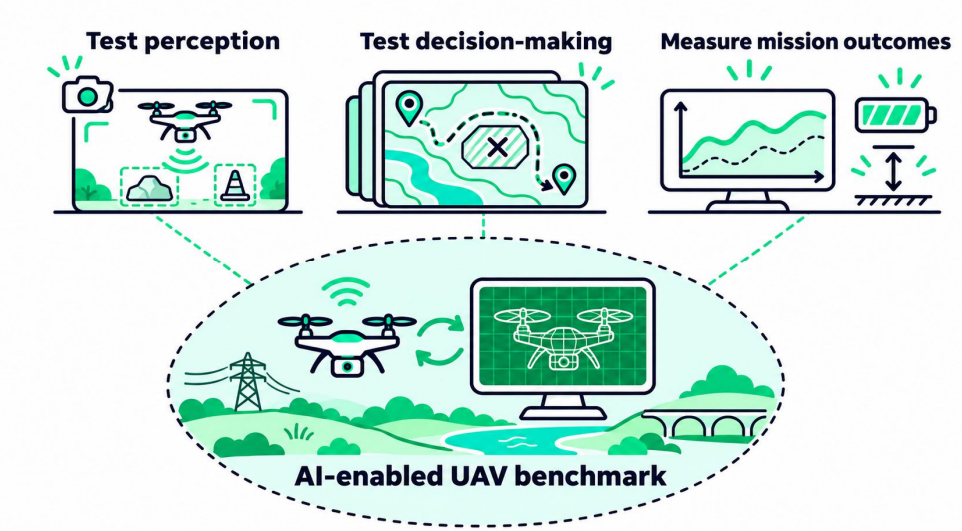


Figure S16.20. A UAV digital twin links perception errors and planning decisions to flight outcomes, showing whether the aircraft completes the survey, reroutes safely, or aborts as intended.

Standardization and Certification Challenges

- **Common standards:** Use shared interface specifications and test methods to compare how aircraft, ground stations, and payloads work together across vendors.
- **Certification evidence:** Map applicable safety and security requirements to test results. Record the hardware, firmware, and operating conditions covered by each test so reviewers can assess where the evidence applies.
- **Change management:** When firmware, an autonomy model, or a payload changes, assess which requirements are affected and repeat the relevant tests. Link the results to the updated configuration.

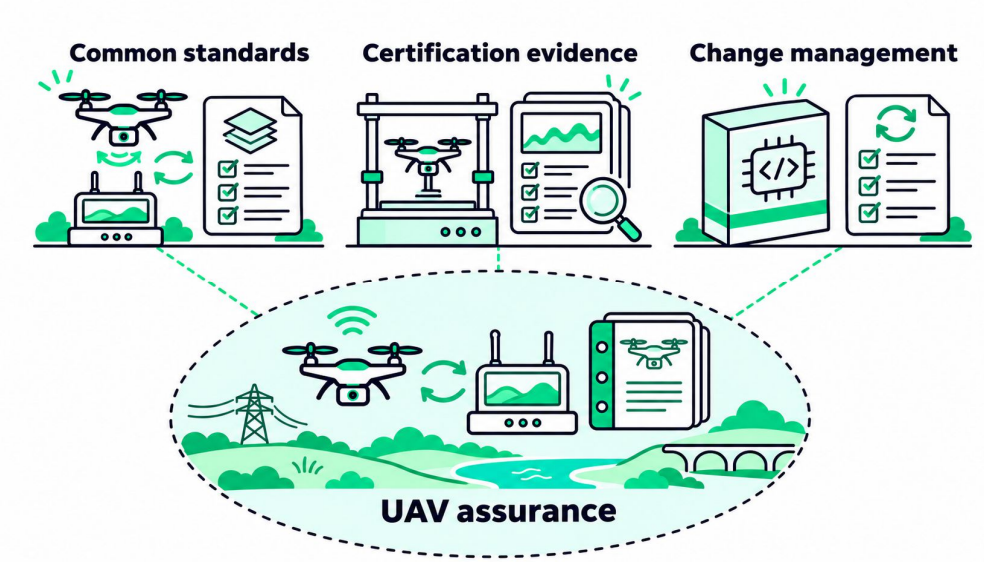


Figure S16.21. Common standards make UAV systems easier to integrate and test. Certification evidence ties those test results to a specific configuration and helps determine what must be retested when it changes.

Fragmentation of Standards and Regulatory Frameworks

- **Different regulatory scopes:** The same UAV may face different requirements depending on the jurisdiction, intended operation, and aircraft configuration.
- **Different test methods:** Two standards can address the same safety objective, such as command-link continuity, while specifying different test conditions, measurement methods, or pass criteria.
- **Evidence mapping:** Map each requirement to the test report and configuration it covers. Identify evidence that can support more than one framework and gaps that need additional analysis or testing.

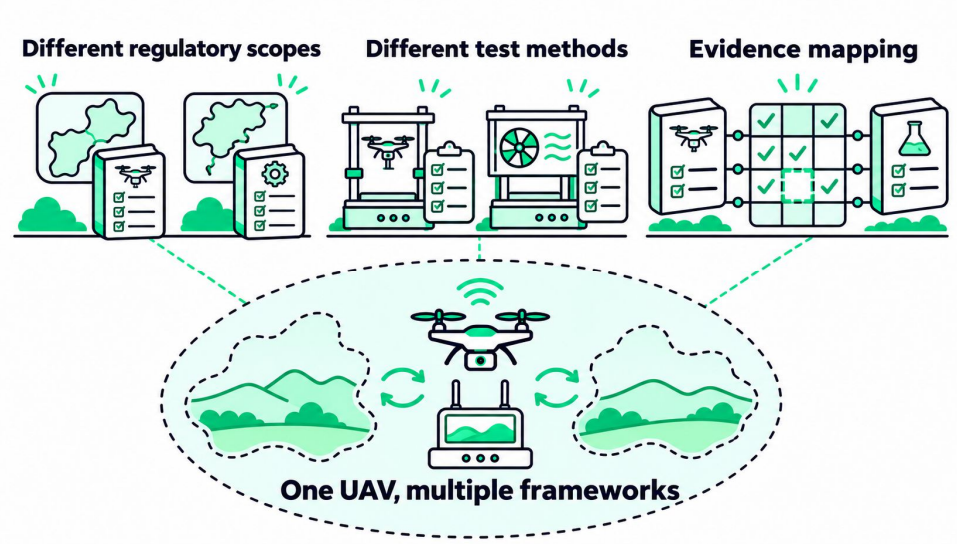


Figure S16.22. The same inspection UAV is evaluated under two regulatory frameworks. A requirement-to-evidence matrix shows where existing test results apply and where differences in test conditions or acceptance criteria leave gaps.

Certification of Autonomy and AI-Enabled UAV Functions

- **Define operating limits:** Specify where and under what conditions the AI function can be used, including visibility, weather, obstacle types, and sensor availability.
- **Validate autonomy:** Combine simulation and representative flight tests to measure missed detections, planning latency, and minimum obstacle clearance. Link each result to the tested model, firmware, and sensor configuration.
- **Verify fallback behavior:** Test how the aircraft responds when sensor data become unreliable or the planner cannot find a valid route. Check that the operator receives a timely alert and that the aircraft executes the planned fallback.

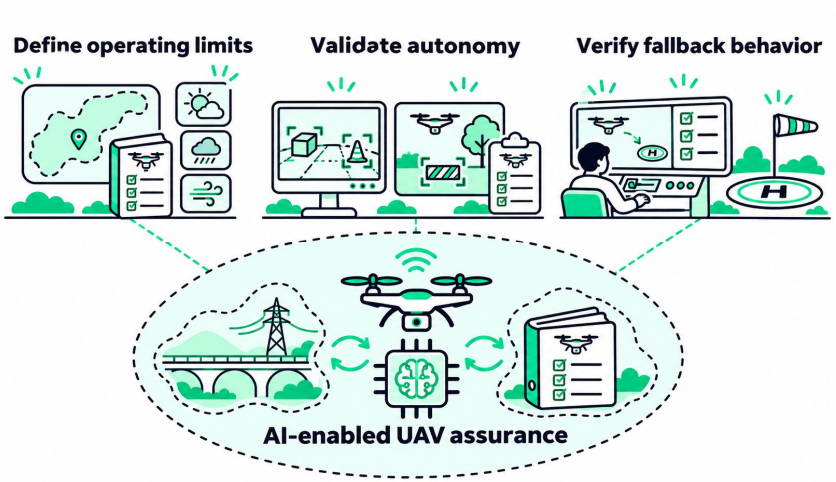


Figure S16.23. An AI-enabled inspection UAV is evaluated within defined operating limits. The evidence covers both normal autonomous flight and the aircraft's response when perception or planning degrades.

Cybersecurity Certification and Airworthiness Security

- **Assess security risks:** Identify how a loss of command-link integrity, unreliable navigation data, or compromised firmware could affect safe flight. Link each credible scenario to a flight hazard and a security requirement.
- **Verify protections:** Test command authentication, signed firmware updates, and contingency behavior under degraded conditions. Record the tested aircraft and ground-station configurations, acceptance criteria, and results.
- **Maintain airworthiness:** Monitor vulnerabilities throughout the UAV's service life. When a patch or configuration change affects a safety-critical function, reassess the associated risks and repeat the relevant tests.



Figure S16.24. Cybersecurity evidence supports UAV airworthiness by tracing threats to flight hazards, documenting how protections were verified, and showing how that evidence is maintained after updates.

BVLOS, Detect-and-Avoid, Remote ID, and UTM Interoperability

- **Detect-and-avoid (DAA):** Evaluate whether the system detects nearby aircraft, identifies a potential conflict, and initiates an avoidance response in time. Measure detection range, alert latency, and achieved separation in representative encounters.
- **Remote ID:** Verify that a compatible receiver can decode the UAV's identification and location messages. Measure reception gaps and data freshness under representative flight conditions.
- **UAS Traffic Management (UTM):** Test whether the operator's system and traffic services interpret flight intent, airspace constraints, and status updates consistently. Include delayed updates and service outages in interoperability tests.

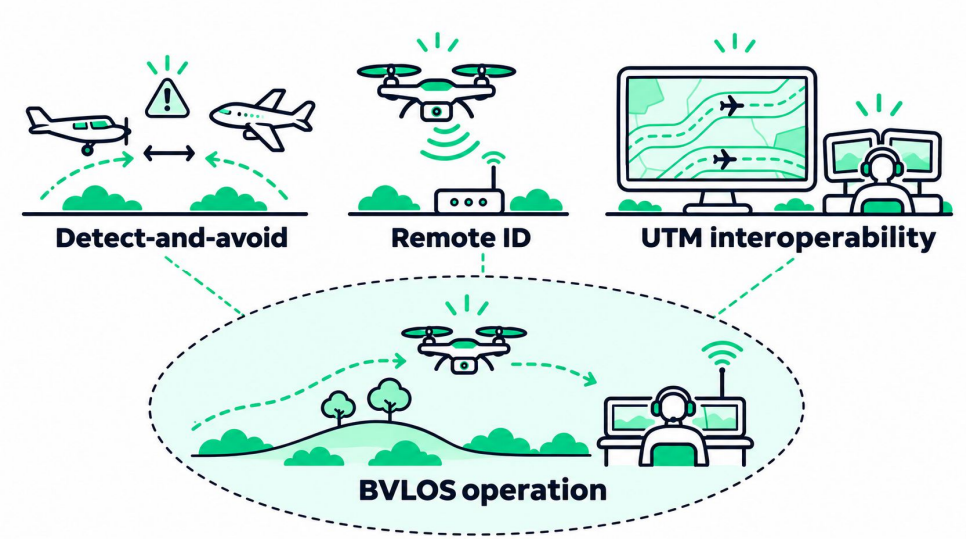


Figure S16.25. A survey UAV flies beyond visual line of sight (BVLOS). Detect-and-avoid addresses immediate traffic conflicts, Remote ID supports identification, and UTM helps coordinate planned operations in shared airspace.

Supply-Chain and Component Trustworthiness Certification

- **Supplier provenance:** Record who supplied each safety-critical component, its part number and revision, and which aircraft received it. Keep the supplier's qualification and acceptance records traceable to the installed part.
- **Component verification:** Verify firmware signatures and check that hardware and software versions match the configuration under assessment. Test the component in the integrated UAV, where its interactions with other systems can affect flight safety.
- **Change control:** When a supplier, part revision, or firmware version changes, assess which safety and security requirements are affected. Repeat the relevant tests and link the results to the revised configuration.

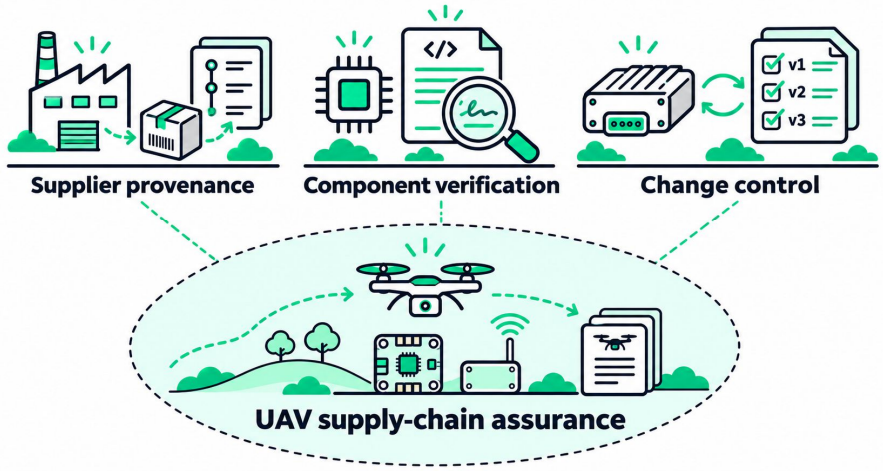


Figure S16.26. UAV supply-chain assurance links each installed component to its origin, verification evidence, and change history. This traceability helps reviewers determine whether the evidence still applies after a component replacement or update.

Testing, Simulation, and Evidence Reproducibility

- **Validate the simulation:** Compare simulated behavior with measured UAV data for the conditions relevant to the requirement. Document where the model agrees with measurements and where discrepancies limit its use.
- **Corroborate with tests:** Use hardware-in-the-loop, bench, and flight tests to check behaviors that depend on the actual aircraft, such as command-link recovery or contingency response. Define pass criteria before testing.
- **Make results reproducible:** Link each result to the requirement it addresses and the exact configuration tested. Preserve software versions, scenario files, random seeds where applicable, raw logs, and analysis scripts so an independent reviewer can repeat the test or verify the reported result.

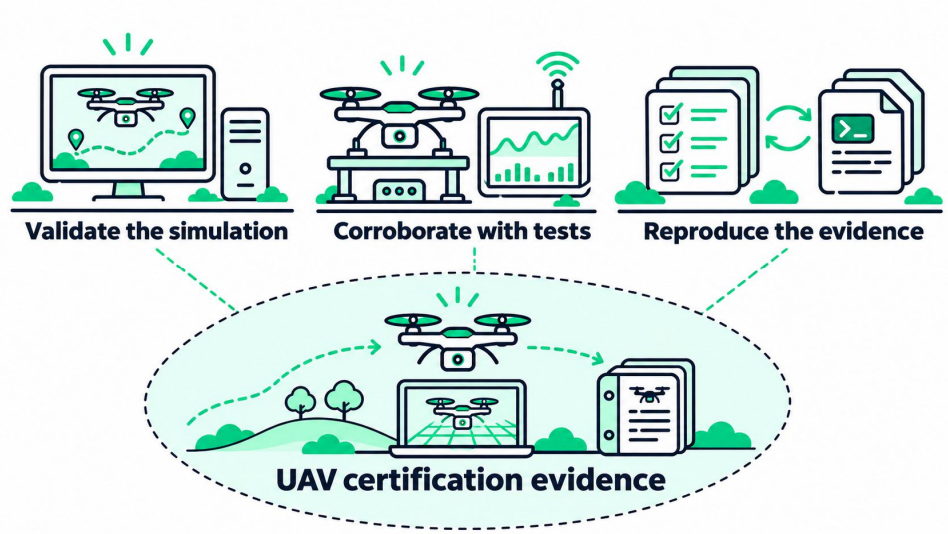


Figure S16.27. Simulation extends scenario coverage, physical tests check how the UAV behaves, and versioned records let reviewers trace each certification claim back to its evidence.